

Ex. 2.2 (5 pts) — Why process creation by a running process is the general case

The first two cases describe what *prompts* a process to be created; they do not describe a different creator. They are not alternatives to the third case, they are the third case seen from the user's side.

When someone double-clicks an icon or types a command, that person never touches the kernel. The click and the keystrokes are input events, and the kernel delivers them to a process that is *already running*: the desktop shell or window manager in the first instance, the command shell in the second. That process interprets the event, decides which program was meant, and calls `fork()` itself. The shell is the creator; the human only supplied the reason. Case 2 differs from case 1 only in where the trigger originated (another process, a timer, an arriving network connection) while the mechanism, a running process calling `fork()`, is unchanged.

All three cases therefore share a single creation primitive and differ only in what caused the running process to invoke it. Unix makes this literal: exactly one process is not born this way, the initial process (`init` or `systemd`, `pid 1`) that the kernel hand-builds at boot. Every other process is the child of some running process.

Ex. 2.3 (5 pts) — A program using `fork()` and `exec()` to run `ls`

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main(void)
7 {
8     pid_t pid = fork();
9
10    if (pid < 0) {                                /* case 1: fork failed */
11        perror("fork");
12        return EXIT_FAILURE;
13    }
14
15    if (pid == 0) {                                /* case 2: we are the child */
16        char *argv[] = { "ls", "-l", NULL };
17        execvp("ls", argv);
18        /* only reached if execvp failed: the image was never replaced */
19        perror("execvp");
20        _exit(127);
21    }
22
23    /* case 3: we are the parent, pid holds the child's pid */
24    int status;
25    if (waitpid(pid, &status, 0) < 0) {
26        perror("waitpid");
27        return EXIT_FAILURE;
28    }
29
30    if (WIFEXITED(status))
```

```

31     printf("parent: child %d exited with status %d\n",
32           (int) pid, WEXITSTATUS(status));
33     else if (WIFSIGNALED(status))
34         printf("parent: child %d killed by signal %d\n",
35               (int) pid, WTERMSIG(status));
36
37     return EXIT_SUCCESS;
38 }

```

`fork()` returns twice, so its return value is the only thing that tells the two processes apart, and all three of its outcomes are handled. A negative value means no child was created. In the child (return value 0) `execvp` replaces the process image with `/usr/bin/ls` while keeping the same pid, searching `PATH` for the binary and passing a NULL-terminated `argv`. A successful `exec` never returns, so the lines after it run only on failure, where they report the error and leave via `_exit` rather than `exit` so the child does not flush the parent's copy of the `stdio` buffers. In the parent (return value the child's pid) `waitpid` blocks until that specific child terminates, which both sequences the output and reaps the child so it does not linger as a zombie.

Compiled with `gcc -Wall -Wextra -o forkexec forkexec.c` (gcc 12.2.0, Debian Linux) and run in a directory holding two small files. The actual output was:

```

total 8
-rw-r--r-- 1 xing xing 1 Sep 21 16:37 alpha.txt
-rw-r--r-- 1 xing xing 2 Sep 21 16:37 beta.txt
parent: child 100 exited with status 0

```

The first three lines come from `ls` after it has taken over the child, the last from the parent after `waitpid` returns. Setting `PATH` to a directory that does not exist exercises the failure branch: `execvp: No such file or directory, then parent: child 107 exited with status 127.`